Design a form - Advanced features

Advanced dynamics

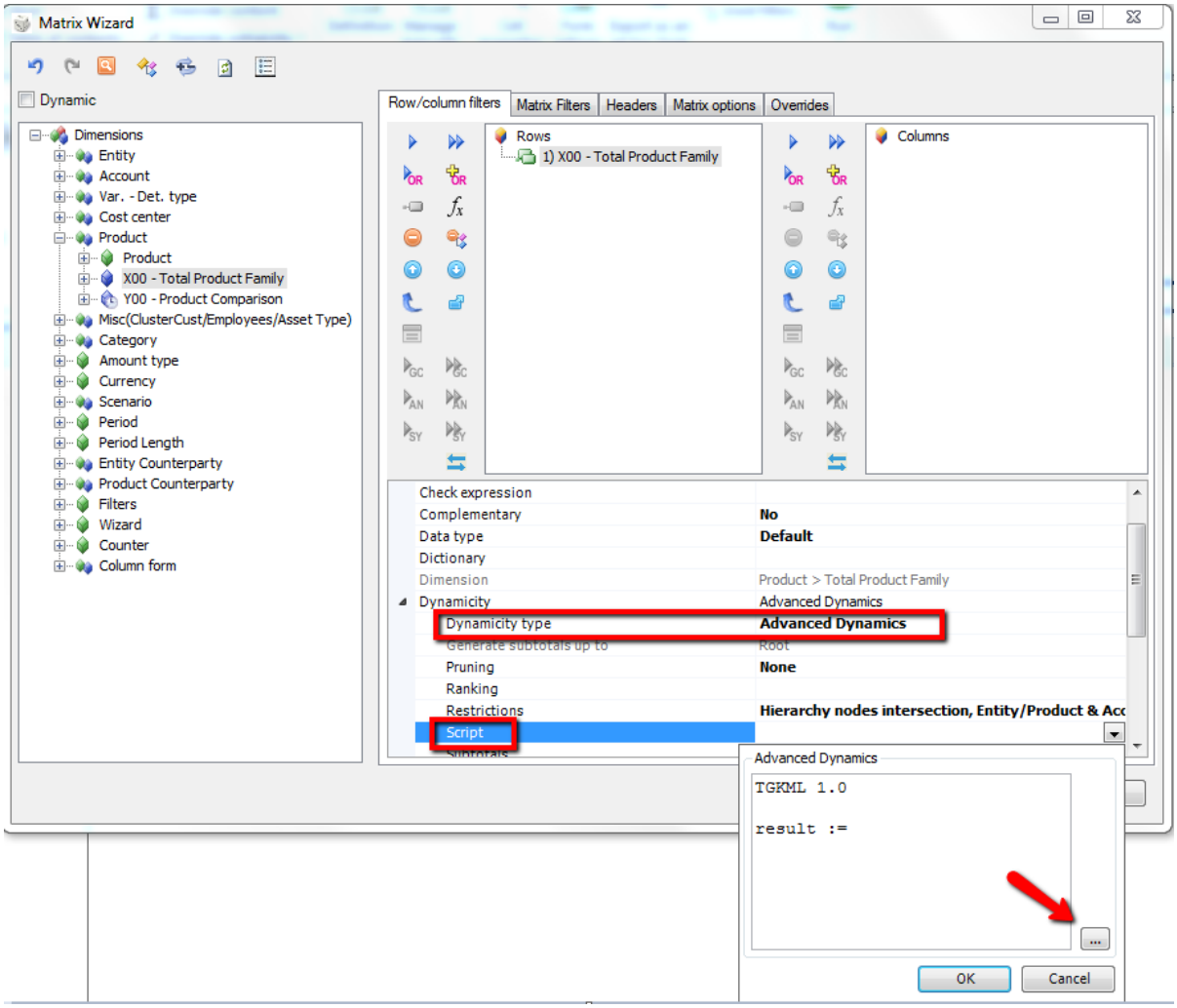
A "standard" matrix is made up of a fixed set of rows and columns defined by the administrator when generating the template. In many cases, a fixed structure may be inefficient to maintain, above all if the elements on the rows or on the columns change often. For example, a matrix displaying on its rows the products may need to be updated once a month, causing an high maintenance cost.

This is why Tagetik Reporting allows the user to define advanced dynamics matrices, namely matrices whose final structure (rows and columns) is defined only at the form run step.

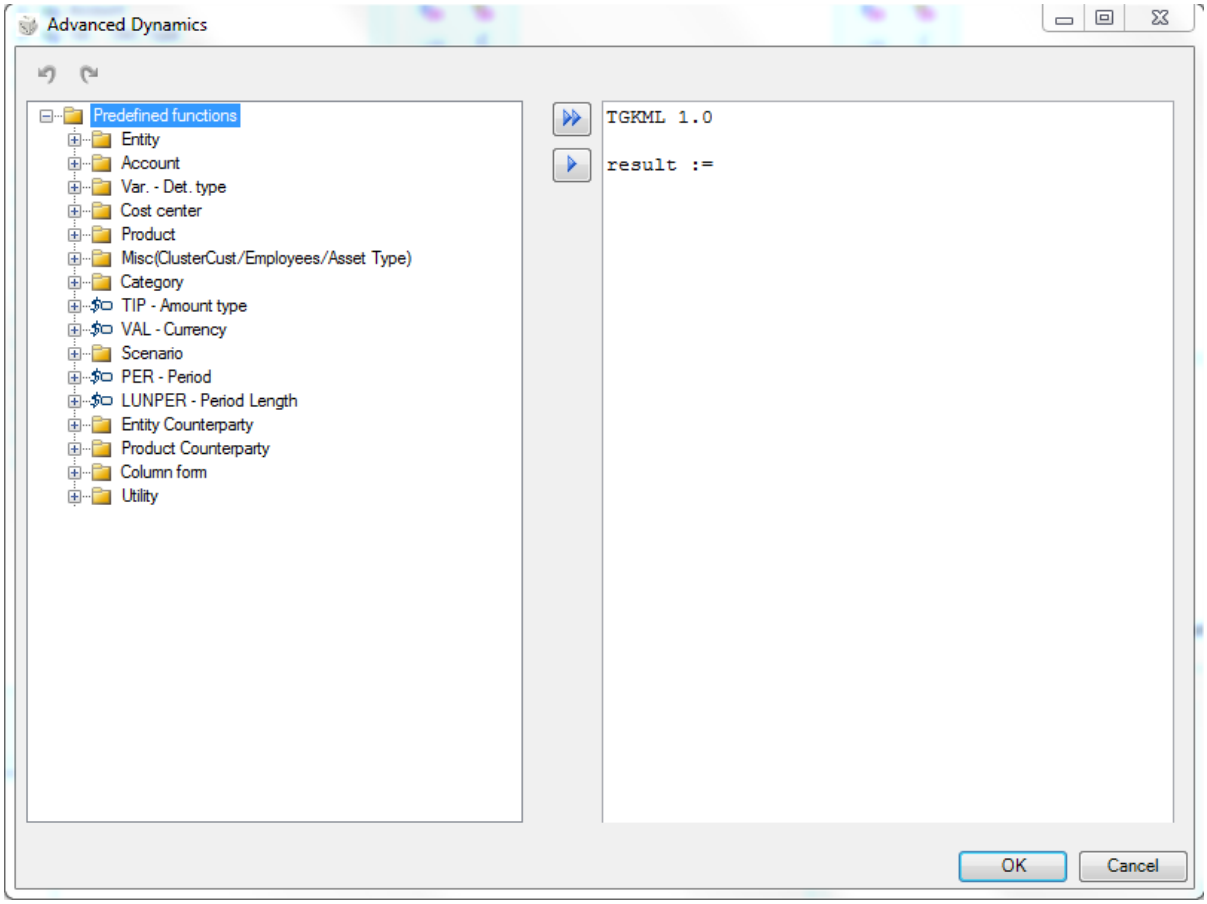
The dynamic structure of a matrix can be defined using the following utilities:

- standard dynamics (for further details see **Dynamicity: details**);
- or using the **Advanced dynamics** option of the **Dynamicity type** property present in the **property grid** of a matrix if the desired dynamic elements cannot be inserted by the standard dynamics (for example the "grandparent" of the current element or all the elements starting with "A"). This option allows to insert, for example:
  - the "grandchild" or "grandparent" nodes or in general one node which is located a random number of levels from the current element;
  - a set of elements that fulfill a given condition: for example, only the elements whose code starts with A;
  - a list of elements sorted according to a given rule which is not by code.

After enabling the "Advanced" dynamicity type, in order to define the elements, it is necessary to specify one script within the **script** property.

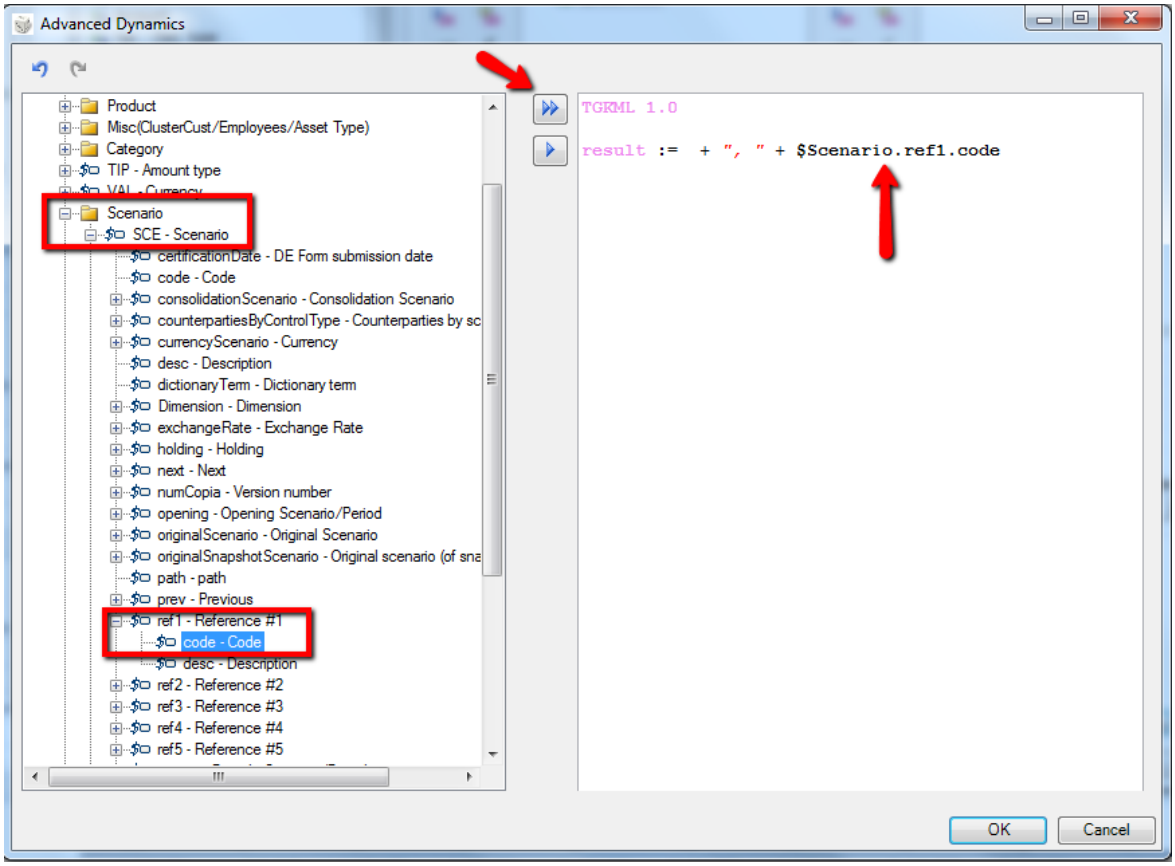


By clicking on the button ... the system opens the **Advanced dynamics** window.

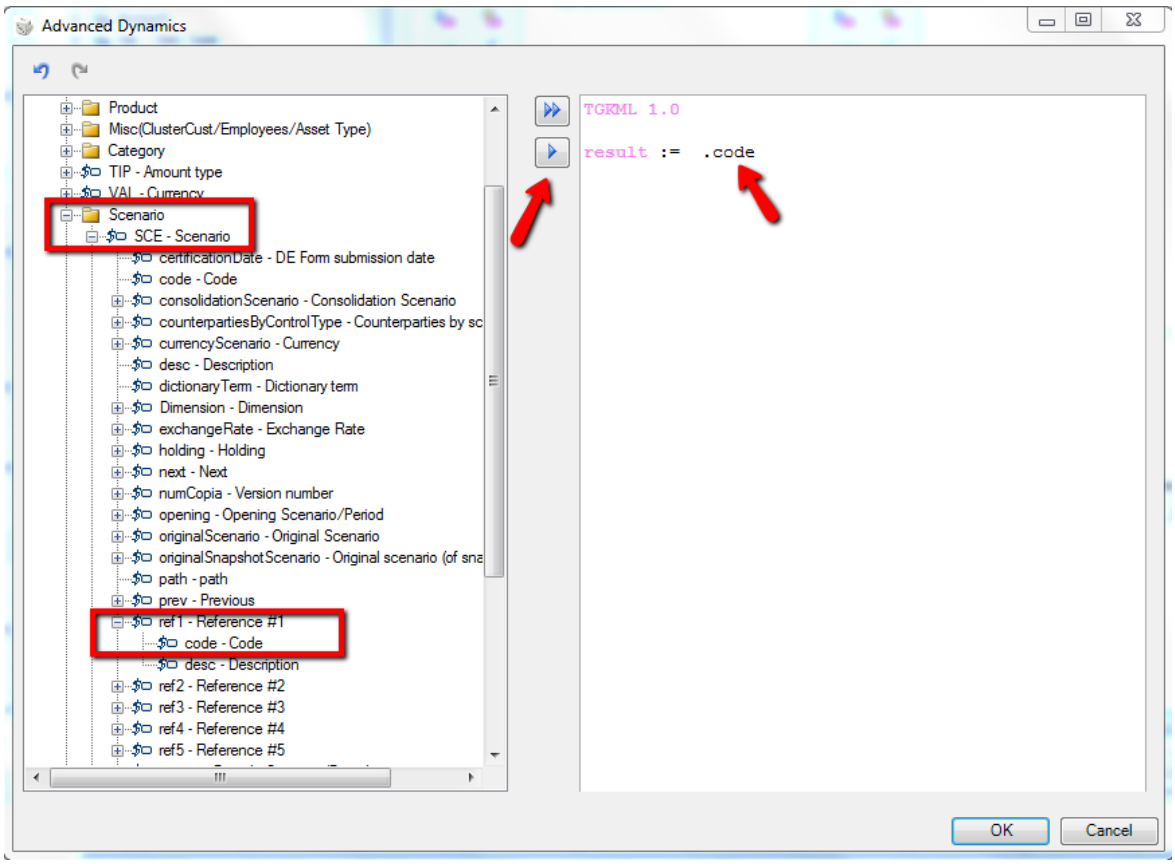


This window is divided in two areas:

- the left area contains all the dimensional attributes (like the ones displayed in the cell fields management) and all the dynamics utilities, where the generic utilities are separated from the utilities related to the table entities;
- the right area contains a text area where to insert the script according to a specific syntax. The user can compose freely the syntax of the script or insert the various elements selecting them from the tree in the right area: he just has to select the desired elements and then click:
  - on the double arrow (). The system will insert the created expression composing all the expressions contained in the parent nodes of the selected node;



- on the single arrow ( ). The system will insert only the expression related to the selected node.



Structure and definition of an advanced dynamics script

An advanced dynamics script:

- starts with the key word "**TGKML 1.0**";
- can contain one or more instructions. Each instruction ends with the character "**;**" and assigns a given content to a variance. This content can be:
  - a string of text;
  - a number;
  - one or more logic functions or operations. This instructions can contain any previously defined classifier;
  - the element of a dimension or node;
  - a set of values (text, numeric or dimension elements)

  The syntax of an instruction is:

  variable_name := content;

  The number of instructions that can be defined in a script is unlimited. All the classifiers are "local" and therefore they can be referred only within the same script. The name of a classifier cannot correspond to the name of the objects belonging to the script language (for example, the dimensions name, the functions name, ...).

  The script syntax is not sensitive to the upper-case letters, therefore it is not possible to define two classifiers whose names differ only for one upper-case or lower-case letter.
- it can contain one or more comments identified by the symbol **//** at the beginning of the row. The characters that follow the symbol // up to the end of the row are ignored. The possible comments can be inserted in any point of the script
- only one classifier is used by the system to get the result of the script. This classifier is predefined in the script and is named **result**. This classifier is mandatory and no other classifier can be defined below. The content of this classifier must be of dimensional type.

Functions / Attributes

Tagetik provides the user with a wide set of functions he can use within an advanced dynamics script. Some functions can be combined with dedicated attributes and other functions can be used in sequence so as to apply one function to the result of the other. The available functions can be grouped as follows:

- **dimensional**
- **not dimensional**
- **position**. These allow to access specific positions within the form. They support the dimensional functions
- **tree navigation**. These allow to navigate the aggregation structures

- **Excel**. As suggested by the name, these are Excel functions that can be used within the TGKML syntax of the script. Generally, these are used to manage the content of a classifier.
- on **sets**.

Result check

In order to analyze the execution steps of an advanced dynamics script and the value the classifiers get in each step, the user has just to click on the button  **Statistics** present both in the Navigation ribbon bar and in the Data entry ribbon bar. The system displays in the **Adv.dyn.scripts** sheet all the scripts present in the form and specifies the matrix in which they have been defined and the value assumed by the various classifiers. In this way, in presence of a very complex script, it is possible to follow the execution process and find possible errors.

Dimensional functions

The dimensional functions return dimensional objects such as the aggregation structures nodes or the last level elements, provided that these objects belong to the filter (row / column, matrix, sheet, form) to which the advanced dynamics has been applied.

In the window where the advanced dynamics script is inserted, the dimensional functions are grouped in nodes, one for each Tagetik dimension.

The symbol "**$**" followed by the name of a dimension represents the function that returns the dimensional object used in the filter (row / column, matrix, sheet, form). For example:

- **var := $Account;** saves in the classifier "var" the employed accounts;
- **var := $Scenario;** saves in the classifier "var" the employed scenarios.

$

Tagetik version. From Tagetik 3.0912

Applicable to All the dimensions

Syntax. *$<DimensionName>[(boolean)]*

Parameters

- *boolean*. (Optional) The possible values are:
  - TRUE. It allows to obtain the lowest level element of the specified dimensional object
  - FALSE. It allows to obtain the specified dimensional object
  - No parameter is equal to TRUE

Resulting value. Filter defined on the <DimensionName> dimension. The names of the usable dimensions are:

- **$Entity.** For the entity dimension
- **$Account.** For the account dimension
- **$Causal.** For the Var. - Det. type dimension
- **$Cust_Dim1...5.** For the custom dimensions
- **$Category.** For the category dimension
- **$AmountType.** For the amount type dimension
- **$Currency.**For the currency dimension
- **$Scenario.** For the scenario dimension
- **$Period.** For the period dimension
- **$PeriodLength.** For the period length dimension
- **$ICEntity.** For the Ctp entity dimension
- **$ICCust_Dim.**For the ctp custom dimension

Example:

`$Entity`

It gives the entity dimension on which it is possible to operate with some attributes

`$Entity.code` or its equivalent `$Entity(TRUE).code`

Let us assume that we have one parameter on one hierarchy of the entity dimension. If at the form run step:

- a lowest level element of the hierarchy is selected,the function returns the code of this element;
- a node of the hierarchy is selected, the function returns one string resulting from the concatenation of the codes of all the lowest level elements related to this node. If the code of the node is required, rather than the list of the lowest level elements, it is necessary to write `$Entity(FALSE).code`

 For each dimensional entity there is one corresponding function without dollar. The expression with dollar (e.g. $Entity) returns the value correctly related to one dimension (e.g. the entity used as tab filter) but the Entity expression written without "$" returns a "descriptor" of the specified dimension, in the "Entity" example, that does not refer to any filter. Therefore:

- **$Entity.desc** returns the string "Tagetik srl"
- **Entity.desc** returns the string "Entity"

These function are used to refer some elements also in the forms where the involved dimension is not employed. For example, let us assume that we want to print one cell field with the description of the entity "A1". The expression **$Entity.getLeaf("A1").-desc** will work only if the system can find a source filter on the entity dimension. The system uses the source filter to identify the dimension on which we are working and looks for the lowest level element with code "A1". If there are no filters on the entity dimension, the expression $Entity does not return anything and it is not possible to search the lowest level element "A1".

On the contrary, the dimension descriptors are always available and return information about all the system dimensions which are not related to the current filters. Thus it is possible to write **Entity.getLeaf("A1").desc**.

In case of hierarchies it is possible to write **hierarchy("G1").Entity.getNode ("N1").desc** or shortly **Entity("G1").getNode("N1").desc**

Hierarchy

Tagetik version. From Tagetik 3.0912

Applicable to All the dimensions on which it is possible to define the hierarchies

Syntax. *Hierarchy(hierarchy_code)*

Parameters

- *hierarchy_code.* Hierarchy code. It must be written between inverted commas.

Resulting value. Filter on the hierarchy with code "code_hierarchy" of the dimension to which the function is applied

Example:

```
$Entity(Hierarchy("G1"))
```

It returns the filter on the G1 hierarchy defined on the entity dimension

Dimensional attribute

Each dimensional function has some attributes that allow to have information related to the dimensional object called by the function itself. The most common attribute is **code**. This attribute is available for almost all the functions and returns a string of text containing the code of the dimensional object called by the function. For example:

- **var := $Account.code;** saves in the "var" classifier the code of the used account;
- **var := $Account.desc;** saves in the "var" classifier the description (in the language related to the user) of the used account

 The dimensional functions and their attributes are widely used in the definition of the matrix headers and in the cell fields.

The attributes can be valid for all the dimensions or specific for a given dimension.

Attributes common to all the dimensions

Attribute 1, ..., Attribute 5

Tagetik version. From Tagetik 3.0912

Applicable to All the dimensions for which it is possible to manage the custom attributes (Entity, Account, Custom Dimensions)

Syntax. *Attribute1*

Resulting value. String containing the value of the custom attribute defined in the table of the dimension it is applied to.

Example:

```
$Entity.Attribute1
```

It returns the attribute 1 value of the entity.

Code

Tagetik version. From Tagetik 3.0912

Applicable to All the dimensional and non dimensional objects

Syntax. *code*

Resulting value. String containing the code of the dimensional object's element

Example:

```
$Entity.code
```

It returns the code of the entity specified in the filter (row / column, matrix, sheet, form)

Desc

Tagetik version. From Tagetik 3.0912

Applicable to All the dimensional and non dimensional objects

Syntax. *Desc([language])*

Parameters

- *language*: (Optional) Code of the table language specified as integer.

Resulting value. String containing the description of the dimensional (and non dimensions) object's element. The optional parameter is supported only in case of multiple descriptions. If missing, the user's table language is used.

Example:

```
$Entity.desc(2)
```

It returns the description of the entity specified in the filter (row / column, matrix, sheet, form), in the table language 2.

DictionaryTerm

Tagetik version . From Tagetik 3.1012

Applicable to All the dimensional objects (lowest level elements and nodes)

Syntax. *DictionaryTerm([language])*

Parameters

- *language*. (Optional) Code of the table language specified as integer.

Resulting value. Possible dictionary term related to the filter (row / column, matrix, sheet, form) or an empty string. The optional parameter is supported only in case if multiple descriptions. If missing, the user's table language is used.

Examples:

```
$Entity.dictionaryTerm
```

It returns the description of the dictionary term in the user's table language related to the filter (row / column, matrix, sheet, form) on the entity.

```
$Entity.dictionaryTerm(2)
```

It returns the description, in the table language 2, of the dictionary term related to the filter (row / column, matrix, sheet, form) on the entity.

Dimension

Tagetik version. From Tagetik 3.0912

Applicable to All the dimensional object (lowest level elements and nodes)

Syntax. *Dimension*

Available attributes

- *code*
- *desc*
- *hierarchyCode*

Resulting value. Dimension or hierarchy to which the filter belongs (row/ column, matrix, sheet, form)

Example:

```
$Entity.dimension.code
```

It returns the "AZI" string that specifies the entity dimension

```
$Entity.dimension.desc
```

It returns the name assigned in the configuration to the entity dimension, for example "Legal Entity"

```
$Entity.dimension.hierarchyCode
```

It returns the code of the hierarchy to which the entity specified in the filter (row / column, matrix, sheet, form) belongs

Link

Tagetik version . From Tagetik 3.1012

Applicable to All the dimensional object (lowest level elements and nodes)

Syntax. *link*

Resulting value. It returns the possible link to the documentation related to the dimensional object's element

Example:

```
$Entity.link
```

It returns the possible link to the documentation related in the table to the entity specified in the filter (row / column, matrix, sheet, form)

LowestReference

Tagetik version. From Tagetik 3.0912

Applicable to All the nodes of a given hierarchy

Syntax. *lowestReference*

Resulting value. Reference lowest level element related to the node

Example:

```
$Entity(Hierarchy("01")).lowestReference.desc
```

It returns the description of the reference entity related to the node used as filter (row / column, matrix, sheet, form) on the hierarchy 01 of the entity.

Path

Tagetik version. From Tagetik 4.2.0

Applicable to All the dimensions and the forms

Syntax. *Path([level], [separator], [cellField])*

Parameters

- *level*. (Optional) The number of levels to go up in the hierarchy (1 = parent, 2 = grandparent, etc...). If missing or zero, it goes up to the root of the hierarchy
- *separator*. (Optional) Separator of each element to use in the returned string. If missing the backslash "\" is used
- *cellField*. (Optional) Cell field expression to evaluate on each node of the hierarchy belonging to the *path*. If missing, the code is used. If the *path*function is applied to the form object, this field is not supported and only the code of the grouping nodes belonging to the *path*is used

Resulting value. String given by the concatenation of the valued expressions on the nodes of the current element's path.

Examples:

Let us assume that there is a filter on the hierarchy "GA - Geographic area" of the entity dimension and that the city of Lucca (LU) is related to the node Tuscany (TUS) and then to the nodes Italy (ITA) / Europe (EU)

```
$Entity.path
```

It returns the string "GA\EUR\ITA\TOS\LU"

```
$Entity.path(0)
```

It returns the string "GA\EUR\ITA\TUS\LU"

```
$Entity.path(2)
```

It returns the string "TUS\LU"

```
$Entity.path(".")
```

It returns the string"GA.EUR.ITA.TUS.LU"

```
$Entity.path(3,".")
```

It returns the string "ITA.TUS.LU"

```
$Entity.path(3,".","$Entity.desc")
```

It returns the string "Italy.Tuscany.Lucca"

Now let us assume that we have a form related to the nodes "DE - Data Entry" \ "PL - Profit & Loss" of the "$" form grouping

```
Report.path
```

It returns the string "$\DE\PL"

```
Report.path(2)
```

It returns the string "DE\PL”

```
Report.path(".")
```

It returns the string "$.DE.PL”

Segment Type

Tagetik version. From Tagetik 3.1012

Applicable to All the hierarchy nodes

Syntax. *segmentType*

Resulting value. Segment type applied to the filter (row / column, matrix, sheet, form)

Example:

```
$Entity(Hierarchy(“01”)).segmentType.desc
```

It returns the description of the segment type related to the nodes used as filter (row / column, matrix, sheet, form) on the hierarchy '01' of the entity

Attributes for the Entity dimension

AdministrativeHQ

Tagetik version. From Tagetik 3.0912

Syntax. *AdministrativeHQ*

Resulting value. Name of the entity's administrative HQ

Example:

```
$Entity.administrativeHQ
```

It returns the entity's administrative HQ

AdministrativeHQBranch

Tagetik version. From Tagetik 3.0912

Syntax. *AdministrativeHQBranch*

Resulting value City of the entity's administrative HQ

Example:

```
$Entity.administrativeHQBranch
```

It returns the city of the entity's administrative HQ

AlternativeCode

Tagetik version. From Tagetik 4.2.1

Syntax. *AlternativeCode*

Resulting value Entity's alternative code

Example:

Let us assume that the entity with code A00 has as alternative code A00_ITA

```
$Entity.alternativeCode
```

It returns the string A00_ITA

BusinessName

Tagetik version. From Tagetik 3.0912

Syntax. *businessName*

Resulting value. Entity business name

Example:

```
$Entity.businessName
```

It returns the entity business name

CertificationDate

Tagetik version. From Tagetik 3.0912

Syntax. *certificationDate*

Resulting value. Entity submission date

Example:

```
$Entity.certificationDate
```

It returns the entity submission date for the process related to the scenario / period present in the filter (row / column, matrix, sheet, form)

ConsolidationPerc

Tagetik version. From Tagetik 3.0912

Syntax. *consolidationPerc*

Resulting value Entity's consolidation percentage

Example:

```
$Entity.consolidationPerc
```

It returns the entity's consolidation percentage.

ConsolidationType

Tagetik version. From Tagetik 3.0912

Syntax. *consolidationType*

Resulting value. Entity consolidation type

Example:

```
$Entity.consolidationType
```

It returns the entity consolidation type

Counterparties

Tagetik version. From Tagetik 3.0912

Stìyntax. *counterparties*

Resulting value Ctp entities related to the entity

Example:

```
$Entity.counterparties.code
```

It returns the codes of the ctp entities related to the entity

CounterpartiesByControlType

Tagetik version. From Tagetik 3.0912

Syntax. *counterpartiesByControlType*

Available attributes

- *code*
- *desc*

Resulting value. Ctp entities related to the current entity for scenario/period (control chart). It requires the presence of a filter (row / column, matrix, sheet, form) on the scenario and period dimensions

 This attribute returns always elements belonging to the "Ctp entity" dimension, therefore the obtained elements can be used directly only within advanced dynamics scripts related to the "Ctp Entity".

Example:

```
$Entity.counterpartiesControlType.code
```

It returns the codes of the ctp entities related to the entity in the control chart

Currency

Tagetik version. From Tagetik 3.0912

Syntax. *currency([scenario],[period])*

Available attributes

- code
- desc

Parameters

- *scenario*. (Optional) Scenario code
- *period*. (Optional) Period code

Resulting value. The reference to currency of the entity specified in the filter (rows / columns, matrix, sheet, form) for the process to which the scenario / period present in the filter (rows / columns, matrix, sheet, form) are related

Examples:

```
$Entity.currency.code
```

It returns the code of the currency related tot he entity specified in the filter (rows / columns, matrix, sheet, form) for the process to which the scenario / period present in the filter (rows / columns, matrix, sheet, form) are related

```
$Entity.currency.code.getLeaf()
```

It returns the filter on the currency related to the entity

```
$Entity.currency(column("MATRIX1",1).$Scenario,column
("MATRIX1",1).$Period.code
```

It returns the code of the currency related to the entity specified in the filter (rows / columns, matrix, sheet, form) for the process to which the scenario / period present in the first column of the matrix "MATRIX1" are related

Cust_dim1Default, Cust_dim2Default, ... Cust_dim5Default

Tagetik version. From Tagetik 4.2.1

Syntax. *cust_dim1Default*

Available attributes

- *code*
- *desc*

Resulting value. Elements of the custom dimension related to the entity specified in the filter (row / column, matrix, sheet, form) read from the table

Example:

```
$Entity.cust_dim1Default.code
```

It returns the code of the custom dimension 1 default element related to the entity specified in the filter (row / column, matrix, sheet, form)

EquityRatioPerc

Tagetik version. From Tagetik 3.0912

Syntax. *equityRatioPerc([scenario],[period])*

Parameters

- *scenario*. (Optional) Scenario code
- *period*. (Optional) Period code

Resulting value. Entity equity ratio %

Example:

```
$Entity.equityRatioPerc
```

It returns the equity ratio %

```
$Entity.equityRatioPerc(column("MATRIX 1",1).$Scenario,column("MATRIX 1",1).$Period)
```

It returns the equity ratio % for the scenario / period specified as parameters of the function and not for those specified in the filter (row / column, matrix, sheet, forms)

ExchangeRate

Tagetik version. From Tagetik 3.0912

Syntax. *exchangeRate([scenario],[period],[entity])*

Parameters

- *scenario*. (Optional) Scenario code
- *period*. (Optional) Period code
- *entity*. (Optional) Entity code

Available attributes

- *average*. Average exch. rate
- *final*. Final exch. rate
- *periodic*. Period average exchange rate

Applicable to. Entity or currency

Resulting value. Currency interest rates of the currency used as filter (row / column, matrix, sheet, form) or of the entity specified as function parameter

Examples:

```
$Currency.exchangeRate.average
```

It returns the entity currency interest rates for the scenario/period/entity defined in the filter (row/column, matrix, sheet, form)

```
$Currency.exchangeRate.final
```

It returns the entity's currency final exchange rate for the scenario/period/entity defined in the filter (row / column, matrix, sheet, form)

```
$Currency.exchangeRate.periodic
```

It returns the entity's average periodic exch. rate for the scenario/period/entity defined in the filter (row / column, matrix, sheet, form)

```
$Currency.exchangeRate(matrix("MATRIX 1").$Scenario, matrix("MATRIX 1").$Period, matrix("MATRIX 1").$Entity).average
```

It returns the entity's currency average exchange rate for the scenario/period/entity defined in the "Matrix 1" matrix

LegalHQ

Tagetik version. From Tagetik 3.0912

Syntax. *legalHQ*

Resulting value. Entity's legal HQ

Examples:

```
$Entity.legalHQ
```

It returns the entity's legal HQ

LegalHQBranch

Tagetik version. From Tagetik 3.0912

Syntax. *legalHQBranch*

Resulting value. City of the entity's legal HQ

Example:

```
$Entity.legalHQBranch
```

It returns the city of the entity's legal HQ

OwnershipPerc

Tagetik version. From Tagetik 3.0912

Syntax. *ownershipPerc([scenario],[period])*

Parameters

- *scenario*. (Optional) Scenario code
- *period*. (Optional) Period code

Resulting value Entity's ownership percentage

Example:

```
$Entity.ownershipPerc
```

It returns the ownership percentage

```
$Entity.ownershipPerc(column("MATRIX 1",1).$Scenario,column("MATRIX 1",1).$Period)
```

It returns the ownership percentage related to the scenario / period specified as function's parameters and not for those specified in the filter (filter / column, matrix, sheet, form)

ProcessStep

Tagetik version. From Tagetik 3.0912

Syntax. *processStep*

Available attributes

- *code*
- *desc*

Resulting value. Step related to the entity for the process to which the scenario / period present in the filter (row / column, matrix, sheet, form) are related

Example:

```
$Entity.processStep.code
```

It returns the code of the step related to the entity present in the filter (row / column, matrix, sheet, form)

ProportionalityPerc

Tagetik version. From Tagetik 3.0912

Syntax. *proportionalityPerc*

Parameters

- *scenario* (Optional). Scenario code
- *period* (Optional). Period code

Resulting value. Entity's proportional percentage

Example:

```
$Entity.proportionalityPerc
```

It returns the entity's proportional percentage

```
$Entity.proportionalityPerc (column ("MATRIX 1",1).$Scenario,column ("MATRIX 1",1).$Period)
```

It returns the entity's proportional percentage for the scenario / period specified as function's parameters and not for those specified in the filter (cow / column, matrix, sheet, form)

Snapshot

Tagetik version. From Tagetik 3.0912

Syntax. *snapshot*

Resulting value. Information related to the Entity/Scenario/Period during the snapshot creation step

Examples:

```
$Entity.snapshot.comment
```

It returns the comment inserted when creating the snapshot for the entity/period/scenario present in the filter (row / column, matrix, sheet, form)

```
$Entity.snapshot.desc
```

It returns the description inserted when creating the snapshot for the scenario/period present in the filter (row / column, matrix, sheet, form)

Classifier

Tagetik version. From Tagetik 3.0912

Syntax. *variable(code)*

Parameters

- *code* . Classifier's code

Available attributes

- *code*
- *desc*

Resulting value. The Classifier element related tot he entity specified in the filter (row / column, matrix, sheet, form)

Example:

```
$Entity.variable("V01").desc
```

It returns the description of the "V01" classifier related to the entity specified in the filter (row / column, matrix, sheet, form)

Attributes for the Account dimension

AccountsToDetail

Tagetik version. From Tagetik 3.0912

Syntax. *accountsToDetail*

Resulting value. Group of accounts to detail (parent accounts) of the control group. In case of:

- **filter on control group** it returns its accounts to detail;
- **filter on account** in a control group, returns the accounts to detail of the control group to which the account belongs

Example:

```
$Account.accountsToDetail
```

It returns all the accounts to detail which are related to the control group specified in the filter (row / column, matrix, sheet, form) or to th control group to which the account specified in the filter (row / column, matrix, sheet, form) belongs

Var. - Det. types Group

Tagetik version. From Tagetik 3.0912

Syntax. *causalsGroup*

Available attributes

- *code*
- *desc*

Resulting value. Var.- Det. types node (if existing) used for creating the control group specified in the filter (row / column, matrix, sheet, form)

Examples:

```
$Account.causalsGroup.code
```

It returns code of the Var. - Det. types node used for creating the control group specified in the filter (row / column, matrix, sheet, form)

```
$Account.causalsGroup.lowest
```

It returns each Var. - Det. type belonging to the Var. - Det. types node used for creating the control group specified in the filter (row / column, matrix, sheet, form)

GroupType

Tagetik version. From Tagetik 3.0912

Syntax. *groupType*

Available attributes

- *code*
- *desc*

Resulting value. Control group type specified in the filter (row / column, matrix, sheet, form) . It type is an object with code and description. The system returns the following values:

- D - Detail
- V - Variation
- B - Balancing
- C - Coherence

In case of:

- **filter on control group** it returns the type of the latter;
- **filter on account** in a control group, it returns the type of the control group to which the account belongs

Examples:

Let us assume that there is a filter on the Variation control group.

```
$Account.groupType.code
```

It returns the string“F”.

```
$Account.groupType.desc
```

It returns the string "Variation" (according to the user's application language)

OriginalAccounts

Tagetik version. From Tagetik 3.0912

Syntax. *originalAccounts*

Resulting value. Set of accounts related to an FST item. In particular, in case of:

- **filter on account**it returns the account itself;
- **FST items** and **nodes of a hierarchy defined on the account dimension**it is equivalent to the children() function and returns the related accounts;
- **formula FST items**it returns the accounts related to all the FST items involved, also indirectly, in the formula (namely, the FST items that depend on FST items that depend on FST items...etc...to which the accounts are related);
- **control group/Var. - Det. type intersection** it returns the account present in the intersection

OriginalControlGroup

Tagetik version. From Tagetik 3.0912

Syntax. *originalControlGroup*

Resulting value. Control group related to a filter (row / column, matrix, sheet, form) on the account. In particular, in case of:

- **filter on control group** it returns the control group;
- **filter on account from control group** it returns the control group to which the account belongs;
- **filter on account from elsewhere** (hierarchies, Financial Statement Templates) it returns all the control groups that detail the account/s

Examples:

```
$Account.originalControlGroup.code
```

It returns the code of the control group to which the account specified in the filter (row / column, matrix, sheet, form) belongs

```
$Account.originalControlGroup.controlGroup
```

It deploys (like the matrix wizard) the control group to which the account specified in the filter (row / column, matrix, sheet, form) belongs

```
$Account.originalControlgroup.causalsGroup.code
```

In case of filter (row / column, matrix, sheet, form) on account, it returns the code of the Var. - Det. types node related to the control group to which the account belongs

Financial Statement Template

Tagetik version. From Tagetik 3.0912

Syntax. *Financial Statement Template*

Available attributes

- code
- desc
- taxonomy

Resulting value. Financial Statement Template to which the account specified in the filter (row / column, matrix, sheet, form) belongs. If the account has been selected from a hierarchy defined on the account dimension, it returns an empty string.

Examples:

```
$Account.schema.code
```

It returns the code of the Financial Statement Template to which the account specified in the filter (row / column, matrix, sheet, form)belongs.

```
$Account.schema.desc
```

It returns the description of the Financial Statement Template to which the account specified in the filter (row / column, matrix, sheet, form) belongs.

ShortDesc

Tagetik version. From Tagetik 3.0912

Syntax. *shortDesc*

Resulting value. Short description in the user's table language of the account specified in the filter (row / column, matrix, sheet, form)

Example:

```
$Account.shortDesc
```

XBRLName

Tagetik version. From Tagetik 3.1012

Syntax. *xbrlName*

Resulting value. Value of the Name XBRL field related to a filter (row / column, matrix, sheet, form) on the account. In particular, in case of:

- **filter on FST item** it returns the value of the Name XBRL field related to the FST item;
- **filter on account from Financial Statement Template**it returns the value of the Name XBRL field related to the FST item to which the account is related;
- in all other cases it returns an error

Example:

```
$Account.xbrlName
```

It returns the value of the Name XBRL field related to the account

Attributes for the Scenario dimension

CertificationDate

Tagetik version. From Tagetik 3.0912

Syntax. *certificationDate*

Resulting value. Entity's submission date

Example:

```
$Entity.certificationDate
```

It returns the entity's submission date for the process to which the scenario / period present in the filter (row / column, matrix, sheet, form) are related.

ConsolidationScenario

Tagetik version. From Tagetik 3.0912

Syntax. *consolidationScenario*

Available attributes

- *code*
- *desc*

Resulting value. If applied to an original scenario, it returns the scenario or the set of consolidation scenarios related to it. If applied to a consolidation scenario it returns the scenario itself.

Example:

```
$Scenario.consolidationScenario.code
```

It returns the code of the consolidation scenario related to the original scenario specified in the filter (row / column, matrix, sheet, form)

CurrencyScenario

Tagetik version. From Tagetik 3.1012

Syntax. *currencyScenario*

Available attributes

- *code*
- *desc*

Resulting values. Currency of the consolidation scenario specified in the filter (row / column, matrix, sheet, form). If applied to an original scenario it returns an empty string.

Example:

```
$Scenario.currencyScenario.code
```

It returns the code of the consolidation scenario specified in the filter (row / column, matrix, sheet, form)

ExchangeRate

Tagetik version. From Tagetik 3.0912

Syntax. *exchangeRate([scenario],[period],[entity])*

Parameters

- *scenario*. (Optional) Scenario code
- *period*. (Optional) Period code
- *entity*. (Optional) Entity code

Available attributes

- *average*. Average exch. rate
- *final*. Final exch. rate
- *periodic*. Period average exch. rate

Applicable to. Entity or currency

Resulting value. Currency interest rates of the currency used as filter (row / column, matrix, sheet, form) or of the entity specified as function's parameter

Examples:

```
$Currency.exchangeRate.average
```

It returns the average exchange rate of the entity's currency for the scenario/period/entity defined in the filter (row / column, matrix, sheet, form)

```
$Currency.exchangeRate.final
```

It returns the entity's final exchange rate for the scenario/period/entity defined in the filter (row / column, matrix, sheet, form)

```
$Currency.exchangeRate.periodic
```

It returns the period average exchange rate of the entity's currency for the scenario/period/entity defined in the filter (row / column, matrix, sheet, form)

```
$Currency.exchangeRate(matrix("MATRIX 1").$Scenario, matrix("MATRIX 1").$Period, matrix("MATRIX 1").$Entity).average
```

It returns the average exchange rate of the entity's currency for the scenario/period/entity defined in the "Matrix 1" matrix

Holding

Tagetik version. From Tagetik 3.0912

Syntax. *holding*

Available attributes

- *code*
- *desc*

Resulting value Holding of the consolidation scenario specified in the filter (row / column, matrix, sheet, form)

Example:

```
$Scenario.holding.code
```

It returns the code of the holding

Next/Prev/Ref1/Ref2/.../Ref5

Tagetik version. From Tagetik 3.0912

Syntax. *next, prev, ref1, ref2, ... ref5*

Available attributes

- code
- desc

Resulting value. Value of the corresponding value read from the scenarios table for the scenario specified in the filter (row / column, matrix, sheet, form). The correspondence attribute - field in the scenario table is:

- **next** - next scenario;
- **prev** - previous scenario;
- **ref1** - reference scenario 1;
- **...**
- **ref5** - reference scenario 5

Examples:

```
$Scenario.next.code
```

It returns the code of the scenario after the scenario specified in the filter (row / column, matrix, sheet, form)

```
$Scenario.prev.code
```

It returns the code of the scenario preceding the scenario specified in the filter (row / column, matrix, sheet, form)

```
$Scenario.ref1.code
```

It returns the code of the reference scenario 1 of the scenario specified in the filter (row / column, matrix, sheet, form)

```
$Scenario.next.next.desc
```

It returns the description of the scenario after the scenario specified in the filter (row / column, matrix, sheet, form)

```
$Scenario.prev.ref1.desc
```

It returns the description of the reference scenario 1 of the scenario preceding the scenario specified in the filter (row / column, matrix, sheet, form)

NumCopia

Tagetik version. From Tagetik 3.1109

Syntax. *numCopia*

Available attributes

- *code*
- *desc*

Resulting value. Snapshot scenario version

Example:

Let us assume that we have an original scenario 2012ACT for which three snapshot scenarios are defined:

- Version 01: 2012ACT_V1
- Version 02: 2012ACT_V2
- Version 03: 2012ACT_V3

If the scenario 2012ACT_V2 is used in the filters (row / column, matrix, sheet, form), the syntax `$Scenario.numCopia` returns the string "2"

Opening

Tagetik version. From Tagetik 4.2.0

Syntax. *opening*

Available attributes

- *CumulatedDays*
- *endDate*
- *MonthDays*
- *startDate*

Resulting value. In presence of a given scenario, it returns the opening scenario / period

Example:

```
$Scenario.opening.startdate
```

It returns the start date of the opening scenario

OriginalScenario

Tagetik version. From Tagetik 3.0912

Syntax. *originalScenario*

Available attributes

- *code*
- *desc*

Resulting value. If applied to a consolidation scenario, it returns the related original scenario. If applied to an original scenario it returns the scenario itself.

Example:

```
$Scenario.originalScenario.code
```

It returns the code of the original scenario related to the consolidation scenario specified in the filter (row / column, matrix, sheet, form)

OriginalSnapshotScenario

Tagetik version. From Tagetik 3.0903

Sintassi. *originalSnapshotScenario*

Available attributes

- *code*
- *desc*

Resulting value. In presence of a given snapshot scenario, it returns the related original scenario

Example:

```
$Scenario.originalSnapshotScenario.code
```

It returns the code of the original scenario related to the snapshot scenario

ScenarioPeriod

Tagetik version. From Tagetik 3.0912

Syntax. *scenarioPeriod*

Available attributes

- *CumulatedDays*
- *endDate*
- *MonthDays*
- *startDate*

Resulting value. Scenario/period related to the filters (row / column, matrix, sheet, form) on the scenario and period dimensions. It can be applied to the scenario and period dimensions.

Example:

```
$Scenario.ScenarioPeriod.endDate
```

It returns the scenario/period end date related to the scenario specified in the filter (row / column, matrix, sheet, form)

```
$Period.ScenarioPeriod.endDate
```

It returns the scenario/period end date related to the period specified in the filter (row / column, matrix, sheet, form)

```
$Period.ScenarioPeriod.(Period.getLeaf("04")).endDate
```

It returns the end date of the (overridden) period with code 04

SnapshotScenarios

Tagetik version. From Tagetik 3.0903

Syntax. *snapshotScenarios*

Available attributes

- *code*
- *desc*

Resulting value. Set of all the snapshot scenarios related to the scenario specified in the filter (row / column, matrix, sheet, form)

Example:

```
$Scenario.snapshotScenario.item(1).code
```

It returns the code of the first snapshot scenario related to the scenario specified in the filter (row / column, matrix, sheet, form)

Version

Tagetik version. From Tagetik 3.0912

Syntax. *version*

Resulting value. Value of the "Version" field read from the scenarios table for the scenario specified in the filter (row / column, matrix, sheet, form)

Example:

```
$Scenario.version
```

It returns the version of the scenario specified in the filter (row / column, matrix, sheet, form)

Year

Tagetik version. From Tagetik 3.0912

Syntax. *year*

Resulting value. Value of the "Year" field read from the scenarios table for the scenario specified in the filter (row / column, matrix, sheet, form)

Example:

```
$Scenario.year
```

It returns the year of the scenario specified in the filter (row / column, matrix, sheet, form)

Attributes for the Period dimension

CertificationDate

Tagetik version. From Tagetik 3.0912

Syntax. *certificationDate*

Resulting value. Entity's submission date

Example:

```
$Entity.certificationDate
```

It returns the entity's submission date for the process to which the scenario / period present in the filter (row / column, matrix, sheet, form) are related

DescNumPeriodo

Tagetik version. From Tagetik 3.1109

Syntax. *descNumPeriod*

Resulting value. Value of the "Period number" field read from the periods table for the period specified in the filter (row / column, matrix, sheet, form)

Esempio:

```
$Period.descNumPeriodo
```

It returns the period number of the period specified in the filter (row / column, matrix, sheet, form)

ExchangeRate

Tagetik version. From Tagetik 3.0912

Syntax. *exchangeRate([scenario],[period],[entity])*

Parameters

- *scenario*. (Optional) Scenario code
- *period*. (Optional) Period code
- *entity*. (Optional) Entity code

Available attributes

- *average*. Average exchange rate
- *final*. Final exchange rate
- *periodic*. Period average exch. rate

Applicable to. Entity or currency

Resulting value. Currency interest rates of the currency used as filter (row / column, matrix, sheet, form) or of the entity specified as function's parameter

Examples:

```
$Currency.exchangeRate.average
```

It returns the average exchange rate of the entity's currency for the scenario/period/entity defined in the filter (row / column, matrix, sheet, form)

```
$Currency.exchangeRate.final
```

It returns the final exchange rate of the entity's currency for the scenario/period/entity defined in the filter (row / column, matrix, sheet, form)

```
$Currency.exchangeRate.periodic
```

It returns the period average exch. rate of the entity's currency for the scenario/period/entity defined in the filter (row / column, matrix, sheet, form)

```
$Currency.exchangeRate(matrix("MATRIX 1").$Scenario, matrix("MATRIX 1").$Period, matrix("MATRIX 1").$Entity).average
```

It returns the average exch. rate of the entity's currency for the scenario/period/entity defined in the "Matrix 1" matrix

Offset

Tagetik version. From Tagetik 3.1109

Syntax. *offset(skip)*

Parameters

- *skip*. Skip from the starting period

Resulting value. Period obtained from the period specified in the filter (row / column, matrix, sheet, form) applied to the specified *skip*. If the skip exceeds the number of periods preceding or after the period specified in the filter, the system considers the relationships between the scenarios creating a chain of periods.

Examples:

Let us assume that we have a filter on Period = April and Scenario = 2013ACT and that the scenario 2012ACT is its previous scenario. Both scenarios consists of 12 periods that represent the months of the year.

```
$Period.offset(-3).code
```

It returns the code of the Period "January"

```
$Period.offset(-5).code
```

It returns the code of the Period "November"

ScenarioOffset

Tagetik version. From Tagetik 3.1109

Syntax. *scenarioOffset(skip)*

Parameters

- *skip*. Skip from the starting period

Resulting value. Scenario obtained from the "skip" of as many periods as specified by the parameter. It returns the scenario specified in the filter (row / column, matrix, sheet, form) if the specified skip does not generate any "skip scenario".

Examples:

Let us assume that there is a filter on Period = April and Scenario = 2013ACT and that the scenario 2012ACT is its previous scenario. Both scenarios consist of 12 periods that represent the months of the year.

```
$Period.scenarioOffset(-6).code
```

It returns the code of the previous scenario namely 2012ACT.

```
$Period.scenarioOffset(-3).code
```

It returns the code of the filter scenario (row / column, matrix, sheet, form) namely 2013ACT.

ScenarioPeriod

Tagetik version. From Tagetik 3.0912

Syntax. *scenarioPeriod*

Available attributes

- *CumulatedDays*
- *endDate*
- *MonthDays*
- *startDate*

Resulting value. Scenario/period related to the filters (row / column, matrix, sheet, form) on the scenario and period dimensions. It can be applied on the scenario and period dimensions.

Examples:

```
$Scenario.ScenarioPeriod.endDate
```

It returns the scenario/period end date related to the scenario specified in the filter (row / column, matrix, sheet, form)

```
$Period.ScenarioPeriod.endDate
```

It returns the scenario/period end date related to the period specified in the filter (row / column, matrix, sheet, form)

```
$Period.ScenarioPeriod.(Period.getLeaf("04")).endDate
```

It returns the end date of the (overridden) period with code 04

Attributes for the column form dimensions

These attributes can be applied to the columns of a transactional form.

CodAttribute

Tagetik version. From Tagetik 3.1109

Syntax. *codAttribute(codAttr)*

Parameters

- *codAttr*. Attribute code

Resulting value. Value of the attribute with code *codAttr* and related to the transactional form column

Example:

```
$Formcolonna.colAttribute("XYZ")
```

It returns the value of the attribute with code "XYZ" related to the current column

Form

Tagetik version. From Tagetik 3.1109

Syntax. *form*

Available attributes

- *code*
- *desc*

Resulting value. Form to which the current column belongs

Example:

`$Formcolumn.form.code`

It returns the code of form to which the current column belongs

Non-dimensional functions

The non-dimensional functions return NON dimensional objects such as the process, the user or the form. The non-dimensional functions have their own attributes.

For example:

- **var := User.code;** Saves in the classifier "var" the code of the user who run the form;
- **var := Form.desc;** Saves in the classifier "var" the description, in the user's table language, of the form in use.

Dataprocessing

Tagetik version. From Tagetik 3.1106

Syntax. *dataprocessing*

Available attributes

- *desc*
- *dataprocessingcount*

Examples:

`dataprocessing.desc`

It returns the "Running data processing"string in the user's application language, in presence of running data processing working on the dimensional set defined by the form filters (row / column, matrix, sheet, form), otherwise it returns an empty string

`dataprocessing.dataprocessingcount`

It returns the number of running data processing on the dimensional set defined by the form filters (row / column, matrix, sheet, form)

Datasource

Tagetik version. From Tagetik 3.1103

Syntax. *datasource*

Available attributes

- *code*
- *desc*

Examples:

```
Datasource.code
```

It returns the code of the data source in use

```
Datasource.desc
```

It returns the description of the data source in use

Process

Tagetik version. From Tagetik 3.0903

Syntax. *process*

Available attributes

- *code*
- *desc*

Example:

```
Process.code
```

It returns the code of the process on which the data entry is performed. If the form has not been run in data entry, this function generates an error

```
Process.desc
```

It returns the description of the process on which the data entry is performed. If the form has not been run in data entry, this function generates an error

Server

Tagetik version. From Tagetik 4.2.1

Syntax. *server*

Available attributes

- url

Example:

```
Server.url
```

It returns the string containing the url of the server to which the client is connected

User

Tagetik version. From Tagetik 3.0903

Syntax. *user*

Resulting value. User who performed the login, namely the current user

Available attributes

- *code*
- *desc*
- *appLang*
  - *code*
  - *desc*
  - *langIndex*
- *tablesLang*
  - *code*
  - *desc*
  - *langIndex*

Examples:

`User.code`

It returns the code of the current user

`User.desc`

It returns the description of the current user

`User.appLang.code`

It returns the code in the application language of the current user

`User.appLang.desc`

It returns the description of the current user's application language

`User.appLang.langIndex`

It returns the integer that identifies the current user's application language

`User.tablesLang.code`

It returns the code of the current user's table language

`User.tablesLang.desc`

It returns the description of the current user's table language

`User.tablesLang.langIndex`

It returns the integer that identifies the current user's table language

Non-dimensional attributes

The non-dimensional functions related to the form and report objects have some attributes that allow to get the information related to these objects.

Form attributes

Code

Tagetik version. From Tagetik 3.0912

Applicable to "Form" non-dimensional objects

Syntax. *code*

Resulting value. String containing the code of the current form

Example:

```
report.code
```

It returns the code of the current form

Desc

Tagetik version. From Tagetik 3.0912

Applicable to "Form" non-dimensional objects

Syntax. *desc([language])*

Parameters

- *language*. (Optional) Code if the table language specified as integer

Resulting value. String containing the description of the current form. The optional parameter is supported only in case of multiple descriptions. If missing, the user's table language is used.

Example:

```
report.desc(2)
```

It returns the description of the current form in the table language 2

Path

Tagetik version. From Tagetik 4.2.0

Applicable to "Form" non-dimensional objects

Syntax. *Path([level], [separator])*

Parameters

- *level*. (Optional) The number of levels to go up in the form grouping (1 = parent, 2 = grandparent, etc...). If missing or equal to zero, it goes up to the root of the grouping
- *separator*. (Optional) Separator of each element to use in the returned string. If missing, the backslash "\" is used

Resulting value. string resulting from the concatenation of the expressions valued on the nodes of the current element's path.

Example:

Let us assume that we have a form related to the nodes "DE - Data Entry" \ "PL - P&L" of the form grouping"$"

```
Report.path
```

It returns the string "$\DE\PL”

```
Report.path(2)
```

It returns the string "DE\PL”

```
Report.path(".")
```

It returns the string "$.DE.PL”

ReportFile

Tagetik version. From Tagetik 4.2.1. (SP20)

Syntax. *reportFile*

Available attributes

- *filename*
- *filepath*
- *fullName*

Resulting value If the current form is an Ad hoc form, it returns some information related to the disk file containing the structure of the form. On the contrary, if the form is shared, it returns an empty string. This function is useful to understand if the current form is Ad hoc or not.

Examples:

This is the syntax to use in order to understand if a form is Ad hoc or not

```
if(report.reportFile.filename=””, “Form shared”, “Form Ad hoc”)
```

`Report.reportfile.filename`

It returns:

- the string "myReport.xlsx" in case of Ad hoc forms "myReport.xlsx";
- an empty string in case of shared form;
- "book1" in case of unsaved Ad hoc form

`Report.reportfile.filepath`

It returns:

- the string "C:\Users\Bob\Documents" in case of Ad hoc form "myReport.xlsx" saved to the user's documents folder;
- an empty string in case of shared form;
- an empty string in case of unsaved Ad hoc form

`Report.reportfile.fullFileName`

It returns:

- the string "C:\Users\Bob\Documents\myReport.xlsx" in case of Ad hoc form "myReport.xlsx" saved to the user's documents folder;
- an empty string in case of shared form;
- "book1" in case of unsaved ad hoc form

RunDate

Tagetik version. From Tagetik 3.0903

Syntax. *runDate([dataFormat],[timeFormat])*

Parameters:

- *dataFormat*: can get one of the following numeric values:
  - -1: no format. the date is not printed
  - 0: the used format is Full
  - 1: the used format is Long
  - 2: the used format is gg-mmm-aaaa
  - 3 (or omitted): the used format is Short
- *timeFormat*: it can get one of the following numeric values:
  - -1 (or omitted): no format. the date is not printed
  - 1: the used format is Full
  - 2: the used format is Long
  - 3: the used format is hh.mm

Resulting value. Date and time at which the form was run. Optionally, it is possible to specify a format type for the date and one for the time. The actual format depends on the user's local settings

Examples:

Let us assume that we have run a form on the 15th of May 2013 at 15.27 and that the user's local setting is "Italian".

```
report.runDate
```

It returns 15/03/13

```
report.runDate(0)
```

It returns Friday 15 March 2013

```
report.runDate(1)
```

It returns 15 March 2013

```
report.runDate(2)
```

It returns 15-mar-2013

```
report.runDate(1,1)
```

It returns 15 March 2013 15.27.59 GMT+01:00

```
report.runDate(1,2)
```

It returns 15 march 2013 15.27.59

```
report.runDate(1,3)
```

It returns 15 March 2013 15.27

```
report.runDate(-1,2)
```

It returns 15.27.59

Report List attributes

Code

Tagetik version. From Tagetik 3.0912

Applicable to "ReportList" non-dimensional object

Syntax. *code*

Resulting value. String containing the code of the form that contains the Report list

Example:

```
ReportList.code
```

It returns the code of the form containing the current Report list

Desc

Tagetik version. From Tagetik 3.0912

Applicable to "ReportList" non-dimensional object

Syntax. *desc([language])*

Parameters

- *language*. (Optional) Code of the table language specified ad integer

Resulting value. String containing the description of the form that contains the Report list. The optional parameter is supported only in case of multiple descriptions. If missing, the user's table language is used. .

Example:

```
ReportList.desc
```

It returns the description of the report containing the current report list

ReportListFile

Tagetik version. From Tagetik 4.2.1 (SP20)

Syntax *ReportListFile*

Available attributes

- *filename*
- *filepath*
- *fullName*

Resulting value. Form containing the current Report list. If the Report list doesn't have any container form as created "on the fly", the system generates an error. It can be used only in the parametric expressions inserted in the Report list creation wizard (for example, the name of the output files/s or the folder's name, the email recipient or the its object or text).

Examples:

```
${reportList.reportListFile.filePath}\myOutput.xlsx
```

In the current example we define as output file of an ad hoc report list the file "myOutput.xlsx" in the same folder of the Report list structure

```
ReportList.reportListfile.filename
```

It returns:

- the string "myReportList.xlsx" in case of ad hoc report list "myReportList.xlsx";
- empty string in case of report list saved into the table;

- the string "book1" in case of unsaved ad hoc report list.

`ReportList.reportListFile.filePath`

It returns:

- "C:\Users\Bob\Documents" in case of ad hoc report list "myReportList.xlsx" saved into the user's documents folder;
- empty string in case of report run in the table;
- empty string in case of unsaved ad hoc report.

`Report.reportfile.fullFileName`

It returns:

- "C:\Users\Bob\Documents\myReportList.xlsx" in case of ad hoc report list "myReportList.xlsx" save into the documents folder;
- empty string in case of report list saved into the table;
- "book1" in case of unsaved ad hoc report list.

Position functions

The position functions allow to access specific positions within a form, for example a row, a column, a matrix.

Often these functions are used in combination with the dimensional functions. For example:

- ***var := row("MATRIX 1", 10). $Entity.code;*** saves in the "var" classifier the code of the entity defined in the row 10 of the matrix "MATRIX 1". This example shows how different functions can be combined together in order to achieve a specific result

Let's see in detail the position functions.

Column

Tagetik version. From Tagetik 3.0912

Syntax. *Column(Matrix code, Column number)*

Parameters

- *Matrix code*. Matrix code. It must be written between inverted commas
- *Column number*. Matrix column number

Resulting value. "columnNumber" column of the matrix with code "matrixCode"

Example:

```
column("MATRIX 1", 10).$Entity.code
```

It returns the code of the entity present in the column 10 of the matrix "MATRIX 1"

Matrix

Tagetik version. From Tagetik 3.0912

Syntax. *Matrix (Matrix code)*

Parameters

- *Matrix code*. Matrix code. It must be written between inverted commas

Resulting value. Reference to the matrix filter of the matrix with code "MatrixCode"

Example:

```
matrix("MATRIX 2").$Entity.code
```

It returns the code of the entity present in the matrix filters of the matrix with code "MATRIX 2"

Parameters

Tagetik version. From Tagetik 3.0912

Syntax. *Parameter(ParameterCode)*

Parameters

- *ParameterCode*. Parameter filter code. It must be written between inverted commas

Resulting value. Reference to the parameter filter with code "ParameterCode"

Example:

```
parameter("S01").$Scenario.code
```

It returns the code of the scenario related to the parameter filter with code "S01"

ParameterValue

Supported version. From Tagetik 3.1012

Syntax. *ParameterValue*

Resulting value. It returns the value of a parameter

Example:

```
parameter("PP").parameterValue
```

It returns the value of the "PP" parameter

PositionIndex

Tagetik version. From Tagetik 3.0912

Syntax. *PositionIndex*

Resulting value Current sheet

Example:

```
positionIndex.$Scenario.code
```

It returns the code of the scenario related to the sheet (for example if the scenario is inserted as tab filter)

PrintFilters

Tagetik version. From Tagetik 4.2.1

Syntax. *printFilters([Position],[cellFiled 1],[cellField 2],...,[cellField N])*

Parameters

- *Position*. (Optional) Reference to a position using the position function (row, column, etc...)
- *cellField 1,..., cellField N*. (Optional) Field attributes list to print for each filter. They must be written between inverted commas

Resulting value. String resulting from the concatenation of the specified field attributes. The separator between two fields of the same filter is " - " and the separator between two filters is ", ".

Examples:

```
Sheet(1).Printfilters
```

It returns the concatenation between the code and the description of the filters related to the first sheet, e.g."A01 - Entity1, 01 - January" (if on the tab filters there are the Entity and Period dimensions)

```
PrintFilters(row("MATRIX1", "5"),"code", "attribute1")
```

It returns the concatenation between the code and the attribute 1 of the filters related to the row 5 of the matrix MATRIX1, e.g."A01 - Entity 1, 01 - January" (if on the row filters there are the Entity and Period dimensions)

Prompt

Tagetik version. From Tagetik 3.0912

Syntax. *Prompt*

Resulting value Reference to the prompt filters for the VBA client .

It is available also for the .NET client

Example:

```
prompt.$Scenario.code
```

It returns the code of the scenario related to the prompt filters

ReportFilters

Tagetik version. From Tagetik 3.1006

Syntax. *reportFilters*

Resulting value. Reference to the form filters for the .NET Reporting Client

Example:

```
reportFilters.$Scenario.code
```

It returns the code of the scenario present in the form filters

Row

Tagetik version. From Tagetik 3.0912

Syntax. *Row(Matrix code, Row number)*

Parameters

- *Matrix code*. Matrix row. It must be written between inserted commas.
- *Row number*. Matrix row number

Resulting value. "Row number" row of the matrix with code "Matrix code".

Example:

```
row("MATRIX 1", 10).$Entity.code
```

It returns the code of the entity defined in the row 10 of the matrix with code "MATRIX 1"

Sheet

Tagetik version. From Tagetik 3.0912

Syntax. *Sheet*

Parameters:

- *sheet*. Sheet number. It must be written between inverted commas

Resulting value. Reference to the "sheet" sheet.

Example:

```
Sheet("13").$Scenario.code
```

It returns the code of the scenario read from the sheet 13

Tree navigation functions

The navigation functions allow to move inside an aggregation structure in order to get a specific object or a set of objects. For example:

- **var := this.lowest;** saves into the "var" classifier the set of lowest level elements of the node on which the dynamicity has been defined. It allows to get the same result as the "Insert lowest level elements" option of the standard dynamics;
- **var := this.lowest.parent;** it saves into the "var" classifier the set of parent nodes of the lowest level elements of the node on which the dynamicity has been defined;
- **var := this.children.onlyLeafs;** saves into the "var" classifier the set of elements related directly to the node on which the dynamicity has been defined.

Parent

Tagetik version. From Tagetik 3.0903

Syntax. *parent*

Resulting value. Parent node of the element or of the node on which the dynamicity has been defined

Example:

```
$Entity.parent.code
```

It returns the code of the parent node of the entity specified in the filter (row / column, matrix, sheet, form)

Children

Tagetik version. From Tagetik 3.0903

Syntax. *children([scenario],[period])*

Parameters

- *scenario*. (Optional) Scenario code
- *period*. (Optional) Period code

Resulting value. Elements and/or children nodes of the node to which it is applied. If the function is applied to an element rather than to a node, it returns the element itself.

Examples:

```
$Entity.children
```

It returns the children elements of the node of the entity hierarchy specified in the filter (row / column, matrix, sheet, form)

```
$Entity.children(2013ACT,03)
```

It returns the children elements of the node of the time dependent entity hierarchy specified in the filter (row / column, matrix, sheet, form) for the period 03 of the scenario 2013ACT

Lowest

Tagetik version. From Tagetik 3.0903

Syntax. lowest(*[scenario],[period],[restrictions]*)

Parameters

- *scenario*. (Optional) Scenario code
- *period*. (Optional) Period code
- *restrictions*. (Optional) Normally, the restrictions "partialize" the result of the deployment according to other filters present in the evaluation. Each restriction can be explicitly enabled or disabled. The syntax to enable or disable the restrictions is the following:

  "nomeLimitazione1:=TRUE|FALSE;nomeLimitazione2:=TRUE|FALSE;…"

  The unspecified restrictions are disabled by default. The available restrictions are:

  - *EntityAccountCustomNLim* (e.g. EntityAccountCustom1Lim, EntityAccountCustom2Lim, …). Restrictions between Entity / Custom Dimension N and between Account / Custom Dimension N
  - *CounterpartyAccountCustom2Lim*. Restrictions between Ctp Entity / Custom Dimension 2 and between Account / Custom dimension 2
  - *IntersectionLim*. Filters intersection

Resulting value. Lowest level elements of the node to which the function is applied. If the function is applied to an element, rather than to a node, it returns the elements itself

Examples:

```
$Entity.lowest
```

It returns the lowest level elements of the node of the entity hierarchy specified in the filter (row / column, matrix, sheet, form)

```
$Entity.lowest(2013ACT,03)
```

It returns the lowest level elements if the node of the time dependent hierarchy specified in the filter (row / column, matrix, sheet, form) for the period 03 of the scenario 2013ACT

```
$Entity.lowest(EntityAccountCustom1Lim:= FALSE;IntersectionLim:=
FALSE)
```

It returns the lowest level elements of the node of the entity hierarchy specified in the filter (row / column, matrix, sheet, form) ignoring the restrictions between the entity and the custom dimension 1. Moreover, it ignores the possible filters on the entity dimension from other hierarchies without intersecting the two sets of filters.

Root

Tagetik version. From Tagetik 3.0903

Syntax. *root*

Resulting value. Root of the hierarchy the node, or the element to which the function is applied, belongs to.

Example:

```
$Entity.root.code
```

It returns the code of the root node of the entity hierarchy specified in the filter (row / column, matrix, sheet, form)

All

Tagetik version. From Tagetik 3.0903

Syntax. *all*

Resulting value All the nodes and the lowest level elements of the dimension present in the filter (row / column, matrix, sheet, form)

Example:

```
$Entity.all.code
```

It returns the codes of all the lowest level elements and the nodes of the entity hierarchy present in the filter (row / column, matrix, sheet, form)

Ancestor

Tagetik version. From Tagetik 3.0903

Syntax. *Ancestor(ancestorLevel)*

Parameters

- *ancestorLevel*. Hierarchy level

Resulting value. Ancestor of level *ancestorLevel*. If the parameter exceeds the level of the hierarchy root, the system returns an empty group

Examples:

```
$Entity.ancestor(1).code
```

It returns the code of the parent node on the entity on which it is applied. Therefore, Ancestor(1) is the parent function

```
$Entity.ancestor(2).code
```

It returns the code of the grandparent node of the entity on which it is applied

AllAncestors

Tagetik version. From Tagetik 3.0903

Syntax. *AllAncestors(ancestorLevel)*

Parameters

- *ancestorLevel*. Hierarchy level

Resulting value. All the ancestor up to the *ancestorLevel*. If the parameter exceeds the level of the hierarchy root, the system returns all the nodes up to the included root

Example:

```
$Entity.allancestors(2).code
```

It returns the code of the parent node and of the grandparent node of the entity on which it is applied

Descendants

Tagetik version. From Tagetik 3.0903

Syntax. *Descendants(descLevel)*

Parameters

- *descLevel*. Hierarchy level

Resulting value. Descendant of level *descLevel*. If the parameter exceeds the level of the hierarchy's lowest level elements, the system returns an empty group

Examples:

```
$Entity.descendants(1).code
```

It returns the codes of the elements and/or children nodes of the entity node on which it is applied. Therefore Descendants(1) is the children function

```
$Entity.descendants(2).code
```

It returns the codes of the elements and/or grandchild nodes of the entity node on which it is applied

AllDescendants

Tagetik version. From Tagetik 3.0903

Syntax. *AllDescendants(descLevel)*

Parameters

- *descLevel*. Hierarchy level

Resulting value All the descendants up to the *descLevel*. If the parameter exceeds the level of the hierarchy's lowest level element, the system returns all the nodes and the descendants lowest level elements up to the included lowest level elements

Example:

```
$Entity.alldescendants(2).code
```

It returns the codes of the elements and/or children and grandchildren nodes of the entity node on which it is applied

Complementary

Tagetik version. From Tagetik 3.0903

Syntax. *Complementary(ancestorLevel)*

Parameters

- *ancestorLevel*. Hierarchy level

Resulting value. Complementary elements in relation to the element present in the filter (row / column, matrix, sheet, form). The complementary is calculated in relation to the ancestor node specified by the parameter *ancestorLevel*

Example:

```
$Entity.complementary(1).code
```

It returns the codes of the entities belonging to the sub-tree of the parent node of the entity present in the filter (row / column, matrix, form, sheet)

Siblings

Tagetik version. From Tagetik 3.0903

Syntax. *siblings*

Resulting value. The brothers of the element to which it is applied, including the element itself. If the source element has to be excluded, it can be "removed", as shown in the second example, from the returned set, using the operator of difference between sets

Examples:

```
$Entity.siblings.code
```

It returns the codes of the element's brothers, including the element itself.

```
$Entity.siblings.code - $Entity.code
```

It returns the codes of the element's brothers, excluding the element.

OnlyLeafs/ OnlyNodes

Tagetik version. From Tagetik 3.0903

Syntax. *onlyLeafs, onlyNodes*

Resulting value. It returns, respectively, only the lowest level elements or only the nodes included in the set

Example:

```
$Entity.children.onlyLeafs
```

It returns, among the children of the current entity node, only the lowest level elements, excluding the nodes

```
$Entity.children.onlyNodes
```

It returns, among the children of the current entity node, only the nodes, excluding the lowest level elements

GetLeaf/ GetNode

Tagetik version. From Tagetik 3.0903

Syntax. *getLeaf(code), getNode(code)*

Parameters

- *code.* Respectively, code of the lowest level element and code of the node to retrieve

Resulting value. Lowest level element or node, with the code specified as parameter, within the same hierarchy of the node to which they are applied

Examples:

```
$Entity.getLeaf("01")
```

It returns, among the lowest level elements of the current entity's hierarchy, only the element with code "01"

```
$Entity.getNode("01").lowest
```

It returns all the lowest level elements of the node with code "01" belonging to the entity on which it is applied

```
$Category.getLeaf("C1")
```

It returns the C1 category

In the groups, a lowest level element could be related to more than one node. In this case the expression returns some copies of the lowest level element for each node it is related to. If only one is needed, it is possible to use the trimSet utility as shown in the following example

```
$Category.getLeaf("C1").trimSet(1).code
```

It returns only the first possibly occurred lowest level element with code "C1"

IsLeaf

Supported version. From Tagetik 3.0903

Syntax. *isLeaf*

Resulting value. 1 if the object the function is applied to is a lowest level element, otherwise 0. The system returns an error if the function is applied to a set with more than one element

Example:

If the entity dimension in the filter is a lowest level element, then the expression

```
$Entity.isLeaf
```

returns 1

Level

Supported version. From Tagetik 3.0903

Syntax. *Level*

Resulting value. Integer that specifies the nesting level (1-based) within the hierarchy. For example, the hierarchy's root will have the level 2, its children the level 2, etc... This function allows to format in different ways the various levels of a given hierarchy with the conditional expressions using the Tagetik style

Example:

If the entity dimension in the filter is a lowest level element belonging to a child node of the hierarchy's root, then the syntax

```
$Entity.level
```

returns the value 3

This

Supported version. From Tagetik 3.0903

Syntax. *this*

Resulting value. Node specified in the row/column filters tree to which the advanced dynamics is applied

Example:

```
this.lowest
```

It returns the lowest level elements of the node used as row/column filter on which the advanced dynamics is defined

Select

Supported version. From Tagetik 3.0903

Synytax. The version has two different syntax:

- *select (field, operator, value)*
- *select (condition)*

Parameters

- *field.* Dimensional attribute (it can be manipulated through the string manipulation utilities) to compare with the *value* parameter.
- *operator.* Comparison operator
- *value.* String or classifier to compare with the *field* parameter
- *condition.* Expression that returns a true/false value for each element of the group the formula is applied to. If the inverted commas have to be used within the string expression, use the QT constant.

Resulting value. Group's elements that fulfill the condition defined by the three parameters (field, operator value) or one parameter (condition)

Examples:

Syntax 1

```
$Entity.lowest.select("$Entity.attribute1", "=", "A")
```

It returns, among the lowest level elements of the source node, only those having "A" as value of the attribute 1

Syntax 2

```
$Entity.children.select("$Entity.lowest.size > 3")
```

It returns, among the children of the source node, only those having more than 3 related lowest level elements

```
$Entity.children.select("LEFT($Entity.code,3)= + QT + "000" +QT)
```

It returns, among the children of the source node, only those whose code starts with "000"

Sort

Supported version. From Tagetik 3.0903

Syntax. sort(*expression, ["DESC"]*)

Parameters

- *expression.* Expression that defines the sorting
- *"DESC"*. (Optional). The default sorting is ascending. When "DESC" is specified as second parameter, the sorting is descending

Resulting value. List of the set's elements sorted according to the *expression* parameter's value

Examples:

```
$Entity.children.sort("$Entity.attribute1")
```

It sorts in ascending order the entity filter's children according to the value of the custom attribute 1

```
$Entity.children.sort("$Entity.attribute1", "DESC")
```

It sorts in descending order the entity filter's children according to the value of the custom attribute 1

```
$Entity.children.sort("$Entity.children.size" , "DESC").sort(
"$Entity.attribute1")
```

It sorts in ascending order the entity filter's children according to the value of the custom attribute 1 and, on equal value of the attribute 1, in descending order according to the number of children of the involved elements

SetEditability

Tagetik version. From Tagetik 3.0903

Syntax. The function has two different syntax:

- *setEditability(editMode, field, operator, value)*
- *setEditability(editMode, condition)*

Parameters

- *editMode*. Edit mode to assign to the group's elements. It can have the following values:
  - "editable" or "e"
  - "protected" or "p"
  - "readonly" or "ro"
- *field*. Dimensional attribute (it can be manipulated through the string manipulation utilities) to compare with the *value* parameter.
- *operator.* Comparison operator
- *value.* String or classifier to compare with the *field*parameter
- *condition.* Expression that returns a true/false result for each element of the group the formula is applied to. If the inverted commas have to be used in the string expression, use the QT constant.

Resulting value. It assigns the edit mode specified for the group's elements that fulfill the condition defined by three parameters (field, operator, value) or one parameter (condition). The editability can be overridden only in accordance with the standard intersection rules, namely:

- If the source node is "not specified" or "editable" in the matrix wizard, using the script it is possible to override any edit mode;
- If the source node is "protected", it can be not editable only;
- If the source node is "not editable", the editability cannot be changed by this function .

Examples:

Syntax 1

```
myAccountsSet.setEditability ("readonly",   "LEFT ($Account.code,3)",
"<=", "000")
```

It makes not editable the accounts whose code starts with "000"

Syntax 2

```
myAccountsSet.setEditability("readonly", "$Account.code.length <= 5")
```

It makes not editable the accounts whose code is made up of less than 5 characters

OrFilter

Supported version From Tagetik 3.0903

Syntax. *orFilter(element1, element2, ... elementN)*

Parameters

- *element1*. Element that will be included in the final filter. It can be defined as a string between inverted commas or as dimensional function
- *element2*. Same as above
- ...

Resulting value. One filter obtained setting to OR the elements defined as parameters. If one of these parameters is a group, all its elements are set to OR.

Example:

```
OrFilter($Entity.getLeaf("01"), $Entity.getLeaf("02"))
```

Set to OR the entities 01 and 02

```
OrFilter($Entity.children)
```

Set to OR the children of the entity hierarchy on which the function is applied

It can be written also as follows: $Entity.children.orFilter()

ContainedFilters

Tagetik version. From Tagetik 3.0903

Syntax. *containedFilters(orFilter)*

Parameters

- *orFilter*. Filter made up of a set of OR filters

Resulting values. opposite function of the Or Filter. It returns the set of OR filters in the filter defined as parameter

Example:

```
$Dest1.containedFilters.lowest.code
```

It gathers all the OR elements and retrieves the lowest level elements

SyntheticSchema

Tagetik version. From Tagetik 3.0903

Syntax. *syntheticSchema(FSTCode)*

Parameters:

- *FSTCode*. FST or FST item code

Valore restituito. Financial statement template or FST item displayed in synthetic mode

Example:

```
SyntheticSchema(@SP)
```

It generates the synthetic FST specified as value of the SP parameter

AnalythicalSchema

Tagetik version. From Tagetik 3.0903

Syntax. *analiticalSchema(FSTCode)*

Parameters:

- *FSTCode*. FST or FST item code

Resulting value. FST or FST item displayed in synthetic mode

Examples:

```
AnalythicalSchema(@SP)
```

It generates the analytic FST specified as value of the SP parameter

ControlGroup

Tagetik version. From Tagetik 3.0903

Syntax. *controlGroup(controlgroupCode)*

Parameters:

- *controlgroupCode*. Code of the control group

Resulting value. Control group with detailed accounts

Example:

```
ControlGroup(@CG)
```

It generates the detail of the control group specified as value of the parameter CG

FromHierarchy

Tagetik version. From Tagetik 3.0903

Syntax. *fromHierarchy(hierarchy_code*)

Parameters.

- *hierarchy_code*. Code of the hierarchy

Resulting value. Same element on which the function is applied but retrieving it from the hierarchy specified as parameter.

Example:

Let us assume that we have the "Geographic are" hierarchy with code "AG" defined on the entity dimension and that the form is filtered on another hierarchy defined on the entity dimension (for example the "01"). Even if the "Geographic area" has not been used within the form, we can get the geographic area of the entity specified as filter.

```
$Entity.fromHierarchy("AG")
```

The system returns the same element used as filter but read from the "AG" hierarchy

If it is a node and in the "AG" hierarchy there isn't any node with the same code, the system signals a warning and the cell is not considered.

```
$Entity.fromHierarchy("AG").parent.code
```

It returns the code of the "AG" hierarchy's node entity specified in the filter (row 7 column, matrix, sheet, form) is related to

Excel functions

As the name suggests, these are Excel functions that can be used in the TGKML syntax of the script. Generally, they are used to manage the classifier's content. For example:

- **var := LEFT($Entity.code, 1);** saves into the classifier "var" the first character of the code of the entity specified in the filter (row / column, matrix, sheet, form).

Logical functions

- *AND(condition1, condition2 ...)* - logical AND
- *OR(condition1, condition2 ...)* - logical OR
- *NOT(condition)* - Negazione
- *IF(condition, TrueValue, FalseValue)*

Aggregation functions

- *SUM(argument1, argument2, ...)*
- *AVERAGE(argument1, argument2, ...)*
- *MAX(argument1, argument2, ...)*
- *MIN(argument1, argument2, ...)*
- *COUNT(argument1, argument2, ...)*
- *PRODUCT(argument1, argument2, ...)*
- *SUBTOTAL (function, argument1, argument2, ...)* Supported only in the row/-column/sheet formulas. Tagetik supports the following function parameter's values:
  - 1 - average
  - 2 - count numbers
  - 4 - max
  - 5 - min
  - 6 - product
  - 9 - sum

Other arithmetical functions

- *SIGN(value)* - It returns -1 if positive value, 1 if negative value
- *ABS(value)* - Absolute value
- *MOD(dividend, divisor)* - Remainder
- *INT(value)* - Floor
- *ROUND(value, figures)* - Rounding

String manipulation

- *CONCATENATE(argument1, argument2, ...)* - String concatenation. It is “String 1” + “String 2” + “String 3” …
- *FIND(text, string, [start])* - Excel "Find" function
- *LEFT(text, charactersNumber)* - It returns the specified number of characters from the beginning of the string
- *LEN(text)* - String length
- *LOWER(text)* - It converts the text to lowercase
- *UPPER(text)* - It converts the text to uppercase
- *MID(text, start, charactersNumber)* - Excel "Mid" function
- *SUBSTITUTE(text, occurrence, substitution)* - Excel "Substitute" function
- *REPLACE(text, start, charactersNumber, substitution)* - Excel "Replace" function
- *RIGHT(text, charactersNumber)* - It returns the specified number of characters from the end of the string
- *REPT(text, times)* - Excel "Rept" function
- *TRIM(text)* - Excel "Trim" function
- *SEARCH(text, string, [start])* - Excel “Find” function but case-insensitive
- *VALUE(text)* - It converts the specified text to numbers. Excel "Value" function

Functions on sets

Some functions, typically the tree navigation functions of the aggregation structures, return sets of elements. It is possible to operate on these sets with specific functions and operators.

And

Supported version. From Tagetik 3.0903

Syntax. * or &

Resulting value. Intersections among sets

Example:

```
$Category.getNode("01").lowest * $Category.getNode("02").lowest
```

It returns the set made up of the lowest level elements related to the nodes "01" and "02" of the categories grouping

Or

Supported version. From Tagetik 3.0903

Syntax. + *or* |

Resulting value. Sets union. The copies are eliminated

Example:

```
$Entity.allAncestors(2) + $Entity.children
```

It returns the set made up of the current node's parent, grandparent and children

Complement

Supported version. From Tagetik 3.0903

Syntax. -

Resulting value. Difference between sets (Complementary set)

Example:

```
$Entity.allDescendants(5) - $Entity.children
```

It returns all the descendants up to the fifth level, excluding the children of the entity node to which it is applied

Contains

Supported version. From Tagetik 3.0903

Syntax. *contains(element)*

Parameter:

- *element*. Element to search in the set. It can be defined as a string between inverted commas or as dimensional function.

Resulting value The value 1 if the element defined as parameter belongs to the set the function is applied to, otherwise the value 0

Example:

```
Entity.lowest.contains($Entity.getLeaf("A01"))
```

It returns 1 if among the descendants of the current entity node there is the entity "A01", otherwise 0

Item

Supported version. From Tagetik 3.0903

Syntax. *item(index)*

Parameters:

- *index*. Identification index of the set's element

Resulting value. Element of the set whose index is specified as parameter (0-based). It returns an error if the set's index is less than +1 children

Example:

```
$Entity.children.item(0)
```

It returns the first child of the entities node

```
$Entity.children.item(1)
```

It returns the second child of the entities node

Set

Supported version From Tagetik 3.0903

Syntax. *set(element1, element2, ... elementN)*

Parameters:

- *element1:* element that will be part of the set. It can be defined as a string between inverted commas or as dimensional function

- *element2* same as above
- ...

Resulting value It creates a set with the elements defined as parameters of the function

Example:

```
Set($Entity.getNode("N1"), $Entity.getNode("N2"))
```

Set containing the nodes N1 and N2 of the entity hierarchy specified in the filter (row / column, matrix, sheet, form)

Size

Supported version. From Tagetik 3.0903

Syntax. *size*

Resulting value. Number of the set's elements

Example:

```
$Entity.children.size
```

It returns the number of the node's children of the entity hierarchy defined in the filter (row / column, matrix, sheet, form)

Sum/Intersect

Supported version From Tagetik 3.0903

Syntax. *sum(set1, set2, ... setN), intersect(set1, set2, ... setN)*

Parameters

- *set1*. set of objects to join or to intersect with the sets specified in the next parameters
- *set2*
- ...

Resulting value. Respectively the union and the intersection between the sets specified as parameters of the function

Example:

Let us consider two classifiers a and b defined as follows:

```
a := Set($Entity.getNode("N1"), $Entity.getNode("N2"));
b := Set($Entity.getNode("N3"), $Entity.getNode("N4"));
sum(a, b)
```

It returns a set containing the nodes N1, N2, N3 and N4 of the entities hierarchy specified in the filter (row / column, matrix, sheet, form)

TrimSet

Supported version From Tagetik 3.0903

Syntax. *trimSet (ElementsNum)*

Parameters:

- *ElementsNum*. Number of elements, starting from the first one, which have to be considered

Resulting value. The first "ElementsNum" of the set. If the set contains less elements, they are all returned.

Example:

```
$Entity.lowest.trimSet(3)
```

It returns the first three entities belonging to the node's sub-tree of the entity hierarchy to which the entity specified in the filter (row / column, matrix, sheet, form) belongs